WEEK1: COUNTERS

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;

    int s =1;

    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

```c
#include <stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int i=1;
    int s=1;
    int c =2;
    while(s <= n)
    {
        i++;
        s+=i;
        c+=3;
     }
     c++;
     printf("%d",c);
}

```

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>

int main(){
    int n;
    scanf("%d",&n);
    int c=1;
    if(n==1)
    {
      //printf("*");
      printf("%d",c);
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
         c+=1;
       for(int j=1; j<=n; j++)
       {
          //printf("*");
          //printf("*");
          c+=3;
          break;
          }
       c++;
     }
     c++;
    }
 printf("%d",c);
}
```

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
   for (i = 1; i <= num;++i)
   {
    if (num % i== 0)
       {
         printf("%d ", i);
       }
    }
 }
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>

int main(){
    int n;
    scanf("%d",&n);
    int c=0;
    for (int i = 1; i<=n;++i){
        c++;
     if (n%i==0){
          //printf("%d ", i);
          c++;
     }
     c++;
     }
     c++;
     printf("%d",c);
}
```

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>

int main(){
    int n;
    scanf("%d",&n);
    int c=1;
    for(int i=n/2; i<n; i++){
        c++;
        for(int j=1; j<n; j = 2 * j){
            c++;
            for(int k=1; k<n; k = k * 2){
                c+=2;
            }
            c++;
        }
        c++;
    }
    c++;
    printf("%d",c);
}
```

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    int rev=0;
    int rem=0;
    int c=0;
    while (n != 0)
    {
        c++;
        rem= n % 10;
        c++;
        rev= rev * 10+rem ;
        c++;
        n/= 10;
        c++;
    }
    c++;
    printf("%d",c+2);
}
```

DIVIDE AND CONQUER

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

```c
#include <stdio.h>
#include <stdlib.h>

int find_first_zero_index(int arr[], int low, int high, int size) {
    if (high < low) {
        return size;
    }

    int mid = low + (high - low) / 2;

    if (arr[mid] == 0 && (mid == 0 || arr[mid - 1] == 1)) {
        return mid;
    }

    else if (arr[mid] == 1) {
        return find_first_zero_index(arr, mid + 1, high, size);
    }

    else {
        return find_first_zero_index(arr, low, mid - 1, size);
    }
}

int divide_and_conquer_zero_count(int arr[], int size) {
    if (size == 0) {
        return 0;
    }

    int first_zero_index = find_first_zero_index(arr, 0, size - 1, size);

    return size - first_zero_index;
}

int main() {
    int m, i;

    if (scanf("%d", &m) != 1) {
        return 1;
    }

    if (m == 0) {
        printf("0\n");
        return 0;
    }
```

```c
int *arr = (int *)malloc(m * sizeof(int));
if (arr == NULL) {
    fprintf(stderr, "Memory allocation failed.\n");
    return 1;
}

for (i = 0; i < m; i++) {
    if (scanf("%d", &arr[i]) != 1) {
        fprintf(stderr, "Input error: Expected %d elements but read only %d.\n
        free(arr);
        return 1;
    }
}

int zero_count = divide_and_conquer_zero_count(arr, m);

printf("%d\n", zero_count);

free(arr);

return 0;
```

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

```c
#include <stdio.h>
#include <stdlib.h>

int find_majority_element(int nums[], int n) {
    int candidate = 0;
    int count = 0;

    for (int i = 0; i < n; i++) {
        if (count == 0) {
            candidate = nums[i];
            count = 1;
        } else if (nums[i] == candidate) {
            count++;
        } else {
            count--;
        }
    }

    return candidate;
}

int main() {
    int n, i;
    if (scanf("%d", &n) != 1) {
        return 1;
    }

    if (n <= 0) {
        return 0;
    }

    int *nums = (int *)malloc(n * sizeof(int));
    if (nums == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            fprintf(stderr, "Input error: Expected %d elements but read only %d.\n", n, i);
            free(nums);
            return 1;
        }
    }

    int majority_element = find_majority_element(nums, n);
    printf("%d\n", majority_element);
    free(nums);

```

Return 0;

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

```c
#include <stdio.h>
#include <stdlib.h>

int find_floor_index_iterative(int arr[], int n, int x) {
    int low = 0;
    int high = n - 1;
    int floor_index = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) {
            return mid;
        } else if (arr[mid] < x) {
            floor_index = mid;
            low = mid + 1;
        } else { // arr[mid] > x
            high = mid - 1;
        }
    }
    return floor_index;
}

int main() {
    int n, i;
    int x;
    if (scanf("%d", &n) != 1) {
        return 1;
    }

    if (n <= 0) {
        return 0;
    }
    int *nums = (int *)malloc(n * sizeof(int));
    if (nums == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return 1;
    }
    for (i = 0; i < n; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            fprintf(stderr, "Input error: Expected %d elements but read only %d.\n", n, i);
            free(nums);
            return 1;
        }
    }
    if (scanf("%d", &x) != 1) {
        fprintf(stderr, "Input error: Could not read value x.\n");
        free(nums);
```

```
            return 1;
        }
    }
    if (scanf("%d", &x) != 1) {
        fprintf(stderr, "Input error: Could not read value x.\n")
        free(nums);
        return 1;
    }
    int floor_index = find_floor_index_iterative(nums, n, x);
    if (floor_index != -1) {
        printf("%d\n", nums[floor_index]);
    } else {
        printf("-1\n");
    }
    free(nums);

    return 0;
}
```

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

```c
#include <stdio.h>
#include <stdlib.h>

int find_pair_recursive(int arr[], int low, int high, int x, int *result_arr) {
    if (low >= high) {
        return 0;
    }
    long long current_sum = (long long)arr[low] + arr[high];

    if (current_sum == x) {
        result_arr[0] = arr[low];
        result_arr[1] = arr[high];
        return 1;
    } else if (current_sum < x) {
        return find_pair_recursive(arr, low + 1, high, x, result_arr);
    } else {
        return find_pair_recursive(arr, low, high - 1, x, result_arr);
    }
}

int main() {
    int n, i;
    int x;
    if (scanf("%d", &n) != 1) {
        return 1;
    }

    if (n <= 1) {
        printf("No\n");
        return 0;
    }

    int *nums = (int *)malloc(n * sizeof(int));
    if (nums == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            fprintf(stderr, "Input error: Expected %d elements but read only %d.\n", n, i);
            free(nums);
            return 1;
        }
    }

    if (scanf("%d", &x) != 1) {
        fprintf(stderr, "Input error: Could not read sum value x.\n");
        free(nums);
```

```c
if (scanf("%d", &x) != 1) {
    fprintf(stderr, "Input error: Could not read sum value x.\n");
    free(nums);
    return 1;
}

int result[2];
int found = find_pair_recursive(nums, 0, n - 1, x, result);

// Print the result
if (found) {
    printf("%d\n", result[0]);
    printf("%d\n", result[1]);
} else {
    printf("No\n");
}
free(nums);
```

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

```c
#include <stdio.h>
#include <stdlib.h>

void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d%s", arr[i], (i == size - 1) ? "" : " ");
    }
    printf("\n");
}

int main() {
    int n, i;
    if (scanf("%d", &n) != 1 || n <= 0) {
        if (n <= 0) {
            return 0;
        }
        return 1;
    }
```

```c
    }
    int *nums = (int *)malloc(n * sizeof(int));
    if (nums == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        return 1;
    }

    for (i = 0; i < n; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            fprintf(stderr, "Input error: Expected %d elements but read only %d.\n", n, i);
            free(nums);
            return 1;
        }
    }

    quickSort(nums, 0, n - 1);
    printArray(nums, n);
    free(nums);

    return 0;
}
```

GREEDY

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

```c
#include <stdio.h>

int main() {
    int V;
    scanf("%d", &V);

    int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int n = sizeof(coins) / sizeof(coins[0]);

    int count = 0;

    for (int i = 0; i < n; i++) {
        if (V >= coins[i]) {
            count += V / coins[i];
            V = V % coins[i];
        }
    }

    printf("%d", count);

    return 0;
}

```

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n, m;
    scanf("%d", &n);
    int g[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &g[i]);

    scanf("%d", &m);
    int s[m];
    for (int i = 0; i < m; i++)
        scanf("%d", &s[i]);

    qsort(g, n, sizeof(int), compare);
    qsort(s, m, sizeof(int), compare);

    int i = 0, j = 0;
    int content = 0;

    while (i < n && j < m) {
        if (s[j] >= g[i]) {
            content++;
            i++;
            j++;
        } else {
            j++;
        }
    }

    printf("%d", content);
    return 0;
}

```

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$. But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

```
3
5 10 7
```

Sample Output

```
76
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int compare_desc(const void *a, const void *b)
{
    int val_a = *(const int *)a;
    int val_b = *(const int *)b;
    if (val_a < val_b) {
        return 1;
    }
    if (val_a > val_b) {
        return -1;
    }
    return 0;
}

int main() {
    int n;
    scanf("%d", &n);
    int calories[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &calories[i]);
    }

    qsort(calories, n, sizeof(int), compare_desc);

    long long total_distance = 0;
    long long power_of_n = 1;

    for (int i = 0; i < n; i++) {
        total_distance += (long long)calories[i] * power_of_n;

        if (i < n - 1) {
            power_of_n *= n;
        }
    }

    printf("%lld\n", total_distance);

    return 0;
}
```

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    qsort(arr, n, sizeof(int), compare);

    long long result = 0;
    for (int i = 0; i < n; i++) {
        result += (long long)arr[i] * i;
    }

    printf("%lld", result);

    return 0;
}

```

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

```c
#include <stdio.h>
#include <stdlib.h>

int asc(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int desc(const void *a, const void *b) {
    return (*(int*)b - (*(int*)a));
}

int main() {
    int n;
    scanf("%d", &n);

    int A[n], B[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &A[i]);

    for (int i = 0; i < n; i++)
        scanf("%d", &B[i]);

    qsort(A, n, sizeof(int), asc);
    qsort(B, n, sizeof(int), desc);

    long long result = 0;
    for (int i = 0; i < n; i++)
        result += (long long)A[i] * B[i];

    printf("%lld", result);

    return 0;
}

```

DP

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*
Output:*6*
Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format

First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    dp[0] = 1;
    if (n >= 1) dp[1] = 1;
    if (n >= 2) dp[2] = 1;
    if (n >= 3) dp[3] = 2;

    for (int i = 4; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 3];
    }

    printf("%lld", dp[n]);

    return 0;
}
```

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3
1 2 4
2 3 4
8 7 1

Output:

15

Explanation:

Totally there will be 6 paths among that the optimal is
Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n
The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

‘

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);
    int grid[n][n];
    long long dp[n][n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &grid[i][j]);

    dp[0][0] = grid[0][0];

    for (int j = 1; j < n; j++)
        dp[j][0] = dp[j-1][0] + grid[j][0];

    for (int j = 1; j < n; j++)
        dp[0][j] = dp[0][j-1] + grid[0][j];

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            long long maxPrev = dp[i-1][j] > dp[i][j-1] ? dp[i-1][j] : dp[i][j-1];
            dp[i][j] = grid[i][j] + maxPrev;
        }
    }

    printf("%lld", dp[n-1][n-1]);

    return 0;
}

```

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1	a	g	g	t	a	b	
s2	g	x	t	x	a	y	b

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

```c
#include <stdio.h>
#include <string.h>

int max(int a, int b) { return (a > b) ? a : b; }

int main() {
    char s1[1000], s2[1000];

    scanf("%s", s1);
    scanf("%s", s2);

    int n = strlen(s1);
    int m = strlen(s2);

    int dp[n+1][m+1];

    for (int i = 0; i <= n; i++)
        dp[i][0] = 0;
    for (int j = 0; j <= m; j++)
        dp[0][j] = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i-1] == s2[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }

    printf("%d", dp[n][m]);

    return 0;
}

```

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

```c
#include <stdio.h>

int upper_bound(int arr[], int size, int x) {
    int left = 0, right = size;
    while (left < right) {
        int mid = (left + right) / 2;
        if (arr[mid] > x)
            right = mid;
        else
            left = mid + 1;
    }
    return left;
}

int main() {
    int n;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    int tail[n];
    int len = 0;
    for (int i = 0; i < n; i++) {
        int pos = upper_bound(tail, len, a[i]);
        tail[pos] = a[i];
        if (pos == len)
            len++;
    }

    printf("%d", len);

    return 0;
}

```

Competitive programming

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int slow = arr[0];
    int fast = arr[0];

    do {
        slow = arr[slow];
        fast = arr[arr[fast]];
    } while (slow != fast);

    slow = arr[0];
    while (slow != fast) {
        slow = arr[slow];
        fast = arr[fast];
    }

    printf("%d", slow);

    return 0;
}

```

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    int slow = arr[0];
    int fast = arr[0];

    do {
        slow = arr[slow];
        fast = arr[arr[fast]];
    }
    while (slow != fast);
    slow = arr[0];
    while (slow != fast) {
        slow = arr[slow];
        fast = arr[fast];
    }

    printf("%d", slow);

    return 0;
}
```

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

```c
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n1, n2;
        scanf("%d", &n1);

        int A[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &A[i]);

        scanf("%d", &n2);
        int B[n2];
        for (int i = 0; i < n2; i++)
            scanf("%d", &B[i]);

        int i = 0, j = 0;
        while (i < n1 && j < n2) {
            if (A[i] == B[j]) {
                printf("%d ", A[i]);
                i++;
                j++;
            } else if (A[i] < B[j]) {
                i++;
            } else {
                j++;
            }
        }

        printf("\n");
    }

    return 0;
}
```

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

- The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

```c
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {

        int n1;
        scanf("%d", &n1);
        int A[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &A[i]);

        int n2;
        scanf("%d", &n2);
        int B[n2];
        for (int i = 0; i < n2; i++)
            scanf("%d", &B[i]);

        int i = 0, j = 0;
        while (i < n1 && j < n2) {
            if (A[i] == B[j]) {
                printf("%d ", A[i]);
                i++;
                j++;
            }
            else if (A[i] < B[j]) {
                i++;
            }
            else {
                j++;
            }
        }

        printf("\n");
    }

    return 0;
}
```

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

```c
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int A[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &A[i]);

    scanf("%d", &k);

    int i = 0, j = 1;

    while (j < n) {
        int diff = A[j] - A[i];

        if (i != j && diff == k) {
            printf("1");
            return 0;
        }
        else if (diff < k) {
            j++;
        }
        else {
            i++;
            if (i == j)
                j++;
        }
    }

    printf("0");
    return 0;
}
```

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

```c
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int A[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &A[i]);

    scanf("%d", &k);

    int i = 0, j = 1;

    while (j < n) {
        int diff = A[j] - A[i];

        if (i != j && diff == k) {
            printf("1");
            return 0;
        }
        else if (diff < k) {
            j++;
        }
        else {
            i++;
            if (i == j)
                j++;
        }
    }

    printf("0");
    return 0;
}

```